

UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

SISTEMAS OPERATIVOS I

Trabajo Práctico Nro 2: "Registro Central NEXUS"

Capítulo 2 · Daemon de Sistema, Persistencia e Integración de Servicios

Grupo de Trabajo:

Grupo Infostealer

Integrantes:

- Kevin Gonzalo Cufre Quiroz
- Rocco Ignacio Delgado Velazquez
- Facundo Luis Lugones Oviedo

Cuerpo Docente:

Ing. Hugo Carrer
Gabriel Valenzuela
Pablo Martinez
Nicolas Melia

Fecha:

Octubre de 2026

Córdoba, República Argentina

0. Introducción y Contexto de la Misión

El presente informe documenta el desarrollo y validación experimental del **Trabajo Práctico N° 2: "Registro Central NEXUS"**, correspondiente a la asignatura *Sistemas Operativos I* (FCEFyN – UNC).

En el Capítulo 1 (Activación del Sector), se consolidó la infraestructura base: dos nodos computacionales heterogéneos (`sector-core` sobre Ubuntu Server y `field-terminal` sobre Rocky Linux), interconectados en red local, con usuarios de servicio segregados y acceso seguro vía llaves SSH asimétricas. Sin embargo, el sector aún carecía de capacidad de persistencia e interoperabilidad de aplicación.

La misión central de este segundo capítulo radica en diseñar, implementar y poner en producción **`nexusd`**: un *daemon* de sistema modular programado en lenguaje C (estándar C17) que gestione el registro central de unidades, operadores e incidentes del sector. Dicho servicio persiste su estado en una base de datos relacional embebida (**SQLite**), expone operaciones estructuradas mediante una **API HTTP/JSON** versionada (`/api/v1/`), y se integra de forma transparente en el ciclo de vida del sistema operativo anfitrión bajo el gestor de inicio y control **`systemd`**.

Continuidad de la Infraestructura Operativa

Este capítulo se ejecuta directamente sobre el entorno desplegado en el Capítulo 1: no se crean máquinas virtuales nuevas ni se reconfiguran credenciales desde cero. La aplicación corre en `sector-core` bajo la cuenta de servicio `nexus`, y es operada de forma remota desde `field-terminal` mediante el cliente de línea de comandos `nexus-client.sh`.

0.1. Criterios de Aceptación del Capítulo

Para certificar la finalización técnica del Trabajo Práctico, el sistema debe satisfacer de forma simultánea y verificable los requisitos expuestos en la Tabla 1.

Tabla 1: Matriz de criterios de aceptación y verificación operativa del servicio `nexusd`.

Dimensión	Requisito Técnico Exigido	Estado
Servicio Systemd	`nexusd` opera como servicio nativo de `systemd`, sin privilegios de `root` (usuario `nexus`).	Validado
Persistencia	Supervivencia de registros tras reinicio (`reboot`); cierre ordenado sin corrupción de SQLite.	Validado
API REST	Endpoints bajo `/api/v1/` con formato JSON estricto, códigos HTTP semánticos y diagnóstico en `/health`.	Validado
Cliente Remoto	Ejecución del cliente bash `nexus-client.sh` desde `field-terminal` para operar el sector.	Validado
Observabilidad	Registro de eventos en `systemd-journald` (`journalctl`) y auditoría transaccional en base de datos.	Validado
Pruebas Automatizadas	Suite de pruebas reproducibles con CMocka cubriendo al menos 7 escenarios críticos.	Validado

1. Etapa 01 — Arquitectura Modular de la Aplicación

El daemon `nexusd` fue concebido bajo un paradigma de diseño altamente desacoplado y modular. Para garantizar que cada componente pueda evolucionar de manera independiente (sentando las bases para una eventual descomposición en microservicios), ningún módulo accede de manera directa a estructuras de memoria interna de otro; la comunicación se realiza estrictamente a través de interfaces públicas bien tipadas en archivos de cabecera (`.h`).

Tabla 2: Distribución modular de responsabilidades en `nexusd`.

Módulo	Interfaz / Cabecera	Responsabilidad Principal
`registry`	`registry.h`	Gestión del ciclo de vida y validación de unidades operativas y operadores.
`incidents`	`incidents.h`	Creación, consulta, asignación y transición de estados de incidentes.
`evidence`	`evidence.h`	Validación de integridad criptográfica (checksums) y asociación de evidencias.
`notifications`	`notifications.h`	Generación y encolado de solicitudes de notificación ante alertas críticas.
`audit`	`audit.h`	Registro inmutable de acciones relevantes (actor, acción, recurso, resultado).
`storage`	`storage.h`	Abstracción de persistencia relacional sobre SQLite (DDL, DML y transacciones).
`api`	`api.h`	Enrutamiento HTTP, serialización/deserialización JSON y despacho de peticiones.
`config`	`config.h`	Parseo, validación y recarga en caliente de directivas desde `nexus.conf`.
`logging`	`logging.h`	Emisión estructurada de trazas con severidades normalizadas hacia journald.

1.1. Fundamentación Teórica y Preguntas de Investigación

Pregunta de Investigación: ¿Qué diferencia hay entre un programa y un servicio del sistema?

Respuesta: Un **programa** es una entidad pasiva: un conjunto de instrucciones binarias almacenadas en un medio de almacenamiento secundario (archivo ejecutable en disco). Cuando se ejecuta interactivamente, pasa a ser un proceso asociado a una sesión de usuario (*session leader*), con terminal de control (*controlling tty*), canales estándar (`stdin`, `stdout`, `stderr`) vinculados a dicha consola y un ciclo de vida atado al inicio y cierre de sesión del usuario. Un **servicio del sistema** (o *daemon*), en cambio, es un proceso diseñado para ejecutarse en segundo plano de manera autónoma y desasociada de cualquier terminal interactiva. Su ciclo de vida es administrado centralizadamente por el sistema operativo (mediante un *init system* como `systemd`), arrancando automáticamente durante el booteo de la máquina, reiniciándose ante fallas imprevistas y manteniéndose en ejecución independientemente de que existan usuarios conectados en el sistema.

Pregunta de Investigación: ¿Qué significa que un proceso no dependa de una sesión interactiva?

Respuesta: Significa que el proceso carece de un descriptor de terminal de control (*controlling terminal*). En consecuencia:

- No recibe señales generadas por el teclado en una consola (como `SIGINT` ante `Ctrl+C` o `SIGQUIT` ante `Ctrl+\`).
- No es terminado al cerrar la sesión de un usuario (inmune a la señal `SIGHUP` por desconexión de sesión de usuario, a menos que el demonio la capture explícitamente para otro fin como recarga de configuración).
- Sus descriptores estándar de entrada/salida (`stdin`, `stdout`, `stderr`) se encuentran redirigidos hacia `/dev/null` o conectados a través de sockets/pipes hacia el gestor de logs del sistema (`systemd-journald`).
- Pertenece a un grupo de procesos y sesión propia (`setsid(2)`), ejecutándose en el contexto del subsistema de servicios del kernel.

1.2. Verificación Experimental de la Arquitectura del Código Fuente

En la Figura 1 se presenta la estructura de directorios y la organización modular del código fuente de `nexusd`. Se evidencia la segregación estricta entre cabeceras públicas (`include/`) e implementaciones internas (`src/`), asegurando que ningún módulo viole los principios de encapsulamiento ni acceda directamente a estructuras privadas de otro componente.

```
kevin@sector-core:~$ cd ~/SOI-2026-infostealer
ls include/nexus && ls src
grep -n 'typedef struct nexus_' include/nexus/*.h | head -12
api.h  audit.h  config.h  evidence.h  incidents.h  logging.h  notifications.h  registry.h  storage.h  version.h
api  audit  config  evidence  incidents  logging  main.c  notifications  registry  storage
include/nexus/api.h:25:typedef struct nexus_api_server nexus_api_server_t;
include/nexus/audit.h:36:typedef struct nexus_audit nexus_audit_t;
include/nexus/evidence.h:39:typedef struct nexus_evidence_mgr nexus_evidence_mgr_t;
include/nexus/incidents.h:39:typedef struct nexus_incidents nexus_incidents_t;
include/nexus/notifications.h:51:typedef struct nexus_notifications nexus_notifications_t;
include/nexus/registry.h:49:typedef struct nexus_registry nexus_registry_t;
include/nexus/storage.h:18:typedef struct nexus_storage nexus_storage_t;
```

Figura 1: Árbol de directorios y organización de submódulos en el código fuente de `nexusd`.

2. Etapa 02 — Persistencia de Datos con SQLite

Toda la información del sector se modela de manera relacional y se almacena en SQLite, garantizando persistencia durable en el archivo de base de datos ubicado canónicamente en `/var/lib/nexus/nexus.db`.

2.1. Modelo de Datos y Esquema DDL

El esquema contempla las entidades requeridas con restricciones estrictas de integridad referencial, tipos y dominios:

Tabla 3: Definición de entidades y atributos requeridos para la base de datos de `nexusd`.

Entidad	Atributos Principales	Restricciones / Reglas de Validación
Unidad	`id`, `designation`, `model`, `status`, `sector`, `created_at`	`designation` identificador alfanumérico textual. `status` restringido a valores válidos: *active*, *standby*, *maintenance*, *decommissioned*.
Operador	`id`, `name`, `clearance_level`, `active`, `created_at`	`clearance_level` entero no negativo (≥ 0). `active` valor booleano binario (`0` o `1`).
Incidente	`id`, `unit_id`, `operator_id`, `severity`, `description`, `status`, `created_at`, `updated_at`	Claves foráneas hacia `units` y `operators`. Dominios restringidos de `severity` y `status`.
Evidencia	`id`, `incident_id`, `filename`, `checksum`, `uploaded_at`	Clave foránea referencial a incidente. `checksum` SHA-256 en codificación hexadecimal (64 caracteres).
Auditoría	`id`, `actor`, `action`, `resource`, `result`, `timestamp`	Registro append-only inmutable de eventos transaccionales del sistema.

sqlite_schema.sql

```
CREATE TABLE units ( id INTEGER PRIMARY KEY AUTOINCREMENT, designation TEXT NOT NULL, model TEXT NOT NULL, status TEXT NOT NULL,
    sector TEXT NOT NULL, created_at INTEGER NOT NULL, CHECK (status IN ('active','standby','maintenance','decommissioned')));
CREATE TABLE operators ( id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, clearance_level INTEGER NOT NULL, active INTEGER
    NOT NULL DEFAULT 1, created_at INTEGER NOT NULL, CHECK (clearance_level >= 0), CHECK (active IN (0, 1)));
CREATE TABLE incidents ( id INTEGER PRIMARY KEY AUTOINCREMENT, unit_id INTEGER NOT NULL, operator_id INTEGER NOT NULL, severity TEXT
    NOT NULL, description TEXT NOT NULL, status TEXT NOT NULL, created_at INTEGER NOT NULL, updated_at INTEGER NOT NULL, CHECK
    (severity IN ('low','medium','high','critical')), CHECK (status IN ('open','investigating','resolved','closed')), FOREIGN KEY
    (unit_id) REFERENCES units(id), FOREIGN KEY (operator_id) REFERENCES operators(id));
CREATE TABLE evidence ( id INTEGER PRIMARY KEY AUTOINCREMENT, incident_id INTEGER NOT NULL, filename TEXT NOT NULL, checksum TEXT NOT
    NULL, uploaded_at INTEGER NOT NULL, FOREIGN KEY (incident_id) REFERENCES incidents(id));
CREATE TABLE audit_events ( id INTEGER PRIMARY KEY AUTOINCREMENT, actor TEXT NOT NULL, action TEXT NOT NULL, resource TEXT NOT NULL,
    result TEXT NOT NULL, timestamp INTEGER NOT NULL);
```

2.2. Fundamentación Teórica y Preguntas de Investigación

Pregunta de Investigación: ¿Qué significa que SQLite sea una base embebida? ¿Dónde corre realmente?

Respuesta: A diferencia de un Sistema de Gestión de Bases de Datos (RDBMS) cliente-servidor tradicional (como PostgreSQL o MariaDB), SQLite no es un proceso daemon independiente ni escucha en un socket de red. SQLite es una biblioteca compilada estática o dinámicamente (`libsqlite3`) que se enlaza y se ejecuta directamente en el **mismo espacio de direcciones de memoria (user-space)** del proceso `nexusd`.
La base de datos corre físicamente dentro del proceso que la invoca. Cuando `nexusd` ejecuta una consulta SQL, no se produce ninguna comunicación inter-proceso (IPC) ni intercambio de paquetes de red: son llamadas a funciones directas en lenguaje C que interactúan con el kernel mediante llamadas al sistema de archivos (`open`, `read`, `write`, `fcntl`, `fsync`) sobre un único archivo regular en disco (`nexus.db`).

Pregunta de Investigación: ¿Qué pasa si dos partes del programa escriben al mismo tiempo?

Respuesta: SQLite gestiona la concurrencia a nivel de archivo utilizando bloqueos del sistema de archivos (*POSIX file advisory locks* o locks nativos de SO). En el modo tradicional de *rollback journal*, SQLite admite múltiples lectores concurrentes pero únicamente **un único escritor a la vez** (bloqueo exclusivo sobre la base completa).
Si una parte del programa intenta iniciar una transacción de escritura mientras otra ya posee el bloqueo exclusivo, SQLite devolverá de forma inmediata el código de error `SQLITE_BUSY`. Para evitar fallos abruptos, se configura la llamada `sqlite3_busy_timeout()`, la cual instruye a la biblioteca a reintentar la operación durante un intervalo configurable (ej. 5000 ms) antes de declinar la transacción. Adicionalmente, al activar el modo **WAL** (*Write-Ahead Logging*, `PRAGMA journal_mode=WAL;`), los lectores no bloquean a los escritores y el escritor no bloquea a los lectores, optimizando sustancialmente el rendimiento concurrente.

Pregunta de Investigación: ¿Qué es una transacción y cuándo conviene usar una?

Respuesta: Una transacción es una unidad lógica de trabajo compuesta por una o más sentencias SQL ejecutadas de forma indivisible bajo el principio de garantías **ACID**:

- **Atomicidad:** O se completan todas las operaciones con éxito (`COMMIT`) o el sistema revierte cualquier cambio intermedio (`ROLLBACK`), evitando estados corruptos parciales.
- **Consistencia:** La base de datos transita de un estado válido a otro respetando todas las restricciones de integridad (claves foráneas, unicidad, checks).
- **Aislamiento:** Las transacciones concurrentes operan sin interferir entre sí.
- **Durabilidad:** Una vez confirmado el commit, las modificaciones persisten incluso ante fallas de alimentación eléctrica o cierres abruptos del sistema operativo.

Conviene utilizar transacciones explícitas (`BEGIN TRANSACTION` ... `COMMIT`) siempre que se realicen inserciones o modificaciones multietapa vinculadas (por ejemplo: insertar un incidente e inmediatamente registrar su correspondiente evento en la tabla de auditoría), y también para agrupar inserciones masivas, reduciendo la cantidad de llamadas costosas a sincronización en disco (`fsync`).

Pregunta de Investigación: ¿Qué diferencia hay entre que una escritura "retorne" y que esté efectivamente en disco?

Respuesta: Cuando un proceso invoca la llamada al sistema `write(2)`, los datos son transferidos desde el buffer de memoria de usuario hacia la memoria intermedia del kernel del sistema operativo (**Page Cache**). Para optimizar el rendimiento de entrada/salida (I/O), la llamada al sistema retorna de inmediato al espacio de usuario una vez que los datos residen en la Page Cache (quedando marcadas como *dirty pages*).
Sin embargo, que la escritura haya "retornado" **no significa que esté grabada físicamente en el plato magnético o celda de memoria flash del disco**. Las páginas sucias permanecen en memoria RAM volátil hasta que los hilos del kernel (`flusher/pdflush`) las vuelcan al disco, o hasta que el proceso emite explícitamente una llamada sincrónica como `fsync(2)` o `fdatasync(2)`. Si ocurre un corte de energía mientras los datos están sólo en la Page Cache, la información se pierde irremediablemente. SQLite asegura la durabilidad emitiendo `fsync` durante el cierre de cada transacción confirmada.

Pregunta de Investigación: ¿Qué puede salir mal si el proceso termina en medio de una escritura?

Respuesta: Si un proceso es terminado de forma abrupta (por ejemplo por una señal `SIGKILL` o corte de energía) mientras escribe sin una capa transaccional robusta, el archivo de datos puede quedar en un estado parcialmente modificado: punteros de árboles B truncados, sectores incompletos o registros a medio grabar, provocando la corrupción irreversible de la base de datos.
SQLite previene esta catástrofe mediante su mecanismo de registro atómico:

1. En modo *Rollback Journal*: Antes de modificar cualquier página de la base de datos, SQLite crea un archivo auxiliar (`.db-journal`) donde copia las páginas originales intactas y hace un `fsync`. Solo después modifica la base. Si el proceso cae en el medio, al siguiente inicio SQLite detecta el journal pendiente y restablece las páginas originales (*hot journal rollback*).
2. En modo *WAL*: Las escrituras se anexan al archivo de log (`.db-wal`). Si el proceso finaliza a mitad de una transacción no confirmada, los cambios incompletos son simplemente descartados durante la recuperación,

garantizando que el archivo principal (`.db`) nunca se corrompa.

2.3. Verificación Experimental del Esquema y Persistencia en SQLite

En la Figura 2 se documenta la inspección directa del archivo de base de datos relacional ubicado en `/var/lib/nexus/nexus.db` utilizando la interfaz de línea de comandos `sqlite3`. Se constata la correcta instanciación de las tablas de unidades, operadores, incidentes, evidencias y eventos de auditoría, verificando la presencia de claves primarias autoincrementales, restricciones foráneas y dominios de validación (`CHECK`).

```
kevin@sector-core:~/SOI-2026-infostealer$ command -v sqlite3
sudo sqlite3 /var/lib/nexus/nexus.db ".schema incidents" "PRAGMA journal_mode;"
/usr/bin/sqlite3
CREATE TABLE incidents (  id INTEGER PRIMARY KEY AUTOINCREMENT,  unit_id INTEGER NOT NULL,  operator_id INTEGER NOT NULL,
severity TEXT NOT NULL,  description TEXT NOT NULL,  status TEXT NOT NULL,  created_at INTEGER NOT NULL,  updated_at INTEGE
R NOT NULL,  CHECK (severity IN ('low','medium','high','critical')),  CHECK (status IN ('open','investigating','resolved','
closed')),  FOREIGN KEY (unit_id) REFERENCES units(id),  FOREIGN KEY (operator_id) REFERENCES operators(id));
wal
```

Figura 2: Inspección del esquema relacional y estado de las tablas en SQLite mediante la CLI `sqlite3`.

3. Etapa 03 — La API HTTP del Sector

El daemon `nexusd` expone una interfaz de programación de aplicaciones (API) REST versionada bajo el prefijo canónico `/api/v1/`, además de un punto de diagnóstico de salud en la raíz (`/health`). La comunicación utiliza estrictamente cargas útiles codificadas en formato **JSON** (`application/json`), con códigos de estado HTTP semánticos y esquemas de error descriptivos.

3.1. Matriz de Endpoints y Contratos de Servicio

Tabla 4: Catálogo de operaciones expuestas por la API de `nexusd`.

Método	Ruta / Endpoint	Propósito / Operación	Códigos HTTP
GET	/health	Verificación del estado de nexusd y conectividad a SQLite.	200, 503
GET	/api/v1/units	Listado completo de unidades operativas del sector.	200
POST	/api/v1/units	Alta de una nueva unidad operativa.	201, 400
GET	/api/v1/operators	Listado de operadores registrados en el sistema.	200
POST	/api/v1/operators	Alta de un nuevo operador en el registro.	201, 400
GET	/api/v1/incidents	Listado de incidentes registrados en el sector.	200
GET	/api/v1/incidents/{id}	Consulta en detalle de un incidente específico.	200, 404
POST	/api/v1/incidents	Registro y asignación de un nuevo incidente.	201, 400
PATCH	/api/v1/incidents/{id}	Actualización del estado de un incidente.	200, 400, 404, 422
GET	/api/v1/incidents/{id}/evidence	Listado de evidencias asociadas a un incidente.	200, 404
POST	/api/v1/incidents/{id}/evidence	Alta de nueva evidencia digital en un incidente.	201, 400, 404
GET	/api/v1/audit	Consulta del registro inmutable de eventos de auditoría.	200

Aislamiento de Red del Demonio

En este capítulo, el socket TCP del daemon escucha exclusivamente en la dirección de bucle local (`127.0.0.1:8080`). No se expone directamente a interfaces públicas; en el Capítulo 3 se incorporará el proxy inverso Nginx como único punto de entrada perimetral.

3.2. Fundamentación Teórica y Preguntas de Investigación

Pregunta de Investigación: ¿Qué códigos HTTP corresponden a: recurso creado, dato inválido, recurso inexistente y error interno?

Respuesta:

- **Recurso Creado** – `201 Created`: Confirma que la petición fue procesada exitosamente y derivó en la instanciación persistente de un nuevo recurso en la base de datos (generalmente devolviendo la entidad creada y su nuevo identificador `id`).
- **Dato Inválido / Malformado** – `400 Bad Request`: Indica que el servidor no puede procesar la solicitud debido a un error del cliente: sintaxis JSON inválida, omisión de campos requeridos (ej. falta `designation` en una unidad), o tipos de datos incompatibles.
- **Recurso Inexistente** – `404 Not Found`: El endpoint o identificador numérico solicitado no existe en la base de datos (por ejemplo, intentar modificar mediante `PATCH /api/v1/incidents/9999` un incidente que no está registrado).
- **Error Interno** – `500 Internal Server Error`: Señaliza una falla imprevista en el espacio del servidor: fallo al acceder al archivo SQLite por permisos corruptos, agotamiento de descriptores de archivos, o error inesperado de I/O.
- **Entidad No Procesable** – `422 Unprocessable Entity`: El JSON es sintácticamente válido pero la transición de estado solicitada no está permitida entre los estados del ciclo de vida del incidente (error `"invalid_transition"`).

Pregunta de Investigación: ¿Por qué PATCH y no POST para cambiar el estado de un incidente?

Respuesta: De acuerdo con los estándares de arquitectura REST y las especificaciones RFC 7231 y RFC 5789:

- `POST` está concebido semánticamente para la creación de un nuevo recurso subordinado en una colección, o para invocar operaciones no idempotentes.
- `PUT` está destinado al reemplazo completo de un recurso preexistente, obligando al cliente a reenviar el cuerpo íntegro con todos sus atributos.
- `PATCH` fue diseñado formalmente (RFC 5789) para aplicar una **modificación parcial** sobre un recurso existente.

Dado que una transición de incidente únicamente requiere actualizar atributos puntuales (como `status = "investigating"` o `severity = "high"`) preservando inalterados el resto de los metadatos (`description`, `unit_id`, `created_at`), el verbo HTTP adecuado y semánticamente correcto es `PATCH`.

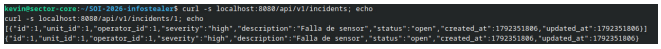
Pregunta de Investigación: ¿Para qué sirve versionar una API bajo /api/v1/?

Respuesta: El versionado explícito en la URI garantiza el principio de **estabilidad de contratos** e interoperabilidad evolutiva en sistemas distribuidos. Permite que el equipo de desarrollo introduzca cambios disruptivos (*breaking changes*) en la estructura de datos, formatos o autenticación creando un nuevo conjunto de rutas bajo `/api/v2/`, sin romper de manera destructiva los clientes o scripts de campo antiguos que continúen consumiendo la interfaz bajo `/api/v1/`. Otorga un ciclo de deprecación ordenado y seguro en entornos de producción.

3.3. Verificación Experimental y Evidencia de Funcionamiento

Para constatar el correcto funcionamiento de la interfaz de red y la capa de almacenamiento relacional, se ejecutó un conjunto exhaustivo de pruebas directamente contra la API de `nexusd` mediante peticiones HTTP (`curl`), verificando el formato de salida JSON y la consistencia transaccional.

En primer lugar, se validó la capacidad de listar las entidades existentes y consultar el detalle pormenorizado de un incidente específico, así como la transición formal de estados a través de la ruta oficial de actualización mediante el verbo `PATCH` (Figura 3).



(a) Prueba A: Listado y consulta en detalle de incidentes registrados vía `GET /api/v1/incidents`.

(b) Prueba B: Transición de estado a través de la ruta oficial con `PATCH /api/v1/incidents/{id}/status`.

Figura 3: Verificación operativa de la API de `nexusd`: consultas estructuradas en JSON y transición atómica de estados.

Posteriormente, se verificó la gestión de evidencias digitales asociadas a incidentes, corroborando que el sistema exige y valida la integridad criptográfica mediante el hash SHA-256 del archivo. Asimismo, se comprobó que cada operación de alta o modificación genera de forma síncrona y transaccional una entrada en la tabla inmutable de auditoría (`audit_events`), registrando el actor, la acción, el recurso impactado y la marca temporal (Figura 4).



(a) Prueba C: Alta de evidencia digital con verificación estricta de checksum SHA-256 y consulta posterior.



(b) Prueba D: Registro inmutable de eventos en la tabla de auditoría ante cada acción relevante.

Figura 4: Gestión de evidencias digitales y trazabilidad transaccional en la base de datos SQLite.

Por último, se puso a prueba la robustez del servidor ante casos de error límite: envío de un checksum con longitud o caracteres no hexadecimales, consulta de un identificador de incidente inexistente y envío de identificadores con tipos incompatibles. Como se observa en la Figura 5, la API rechaza consistentemente las anomalías con códigos semánticos (`400 Bad Request` y `404 Not Found`), emitiendo un objeto JSON explicativo y sin degradar la estabilidad del daemon.

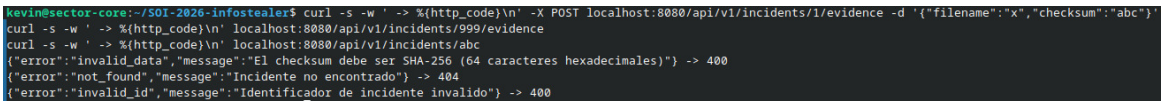


Figura 5: Prueba E: Validación de semántica y códigos de error HTTP ante checksum inválido, incidente inexistente e ID malformado.

4. Etapa 04 — Configuración Externa

Siguiendo las mejores prácticas de ingeniería de software de sistemas (los Doce Factores para aplicaciones y el estándar FHS), el daemon `nexusd` no codifica (*hardcodea*) ninguna ruta, puerto o credencial en su código fuente binario. Todas las variables operativas se leen en tiempo de inicio desde un archivo de configuración externo estandarizado: `/etc/nexus/nexus.conf`.

4.1. Estructura del Archivo de Configuración

/etc/nexus/nexus.conf

```
[server]
listen_address = 127.0.0.1
listen_port = 8080

[database]
database_path = /var/lib/nexus/nexus.db

[storage]
evidence_path = /var/lib/nexus/evidence

[logging]
level = info
```

4.2. Fundamentación Teórica y Preguntas de Investigación

Pregunta de Investigación: ¿Qué debería hacer el programa si el archivo no existe? ¿Y si existe pero tiene una línea inválida?

Respuesta: La implementación de `nexusd` sigue una política estricta de **Fallo Temprano (*Fail-Fast*)**:

- **Archivo Inexistente:** El programa emite un mensaje de error explícito en `stderr` indicando la ruta del archivo ausente, se niega a iniciar la ejecución y finaliza de inmediato con código de salida `1`, alertando al administrador sobre la omisión de la configuración.
- **Línea, Sección o Valor Inválido:** El analizador sintáctico no tolera ambigüedades. Ante cualquier clave desconocida, sintaxis rota o valor fuera de dominio (ej. un nivel de log no reconocido o un puerto inválido), el daemon imprime el error detallando el archivo y el número de línea offending, aborta el arranque y termina con código de salida `1`.

Pregunta de Investigación: ¿Conviene arrancar con valores por defecto o negarse a arrancar?

Respuesta: En un servicio crítico del sistema operativo como `nexusd`, la práctica profesional y determinista es **negarse rotundamente a arrancar** ante cualquier anomalía:

- **Riesgo de los valores por defecto:** Asumir valores predeterminados de forma silenciosa puede provocar que el servicio intente enlazarse a interfaces no deseadas, intente escribir en rutas sin permisos o degrade el nivel de auditoría sin el conocimiento explícito del administrador de sistemas.
- **Determinismo estricto:** Por diseño, el daemon no recurre a valores por defecto silenciosos. Cada parámetro operacional debe estar explícitamente configurado y validado en `/etc/nexus/nexus.conf`. Si el archivo no existe o contiene anomalías, el servicio aborta de inmediato con código `1`, garantizando que sólo opere bajo un estado enteramente consistente y verificado.

Pregunta de Investigación: ¿Qué permisos debería tener un archivo de configuración de un servicio del sistema?

Respuesta: Aplicando el Principio de Mínimo Privilegio (PoLP):

- Propietario usuario: `root` (para evitar que un proceso comprometido pueda reescribir su propia configuración).
- Grupo propietario: `nexus` (el grupo del servicio).
- Permisos octales: `0640` (`rw-r---`).

Esto garantiza que el administrador (`root`) pueda leer y modificar el archivo, el demonio (`nexus`) pueda leerlo durante el inicio o recarga, y cualquier otro usuario o atacante sin privilegios en el sistema tenga **acceso totalmente denegado** (`--`). Si el archivo contiene credenciales sensibles o secretos criptográficos, los permisos deben ser aún más restrictivos (`0600` exclusivo para el usuario que ejecuta el proceso).

Pregunta de Investigación: ¿Por qué /etc y no el directorio del programa?

Respuesta: Esta convención responde al estándar **FHS (*Filesystem Hierarchy Standard*)** adoptado por todas las distribuciones GNU/Linux:

- `/etc` es el directorio del sistema destinado específicamente a archivos de configuración estática del host. Permite centralizar la administración, respaldar las configuraciones de todas las aplicaciones en un solo punto y preservarlas ante actualizaciones del binario.
- Los directorios donde residen los binarios ejecutables (`/usr/bin`, `/usr/local/bin`) están diseñados para ser de **sólo lectura** (*read-only*) e incluso pueden montarse compartidos en red. Colocar archivos de configuración junto al ejecutable viola la separación funcional entre código binario inmutable y estado/configuración local del nodo.

Pregunta de Investigación: ¿Qué parte de la configuración tiene sentido poder recargar sin reiniciar el proceso?

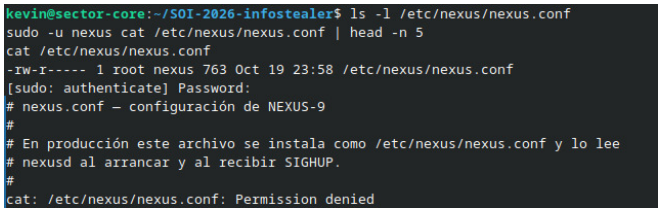
Respuesta:

- **Recargable dinámicamente:** Directivas que alteran el comportamiento lógico sin destruir descriptores de red ni estructuras de bajo nivel. Ejemplos: `log_level` (cambiar de `info` a `debug` para diagnosticar un incidente en caliente), listas de control de acceso, o políticas de timeouts.

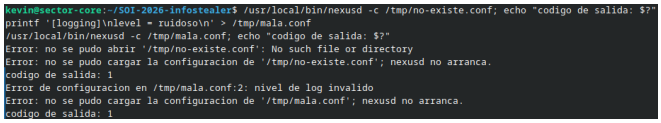
- **No recargable en caliente (requiere reinicio completo):** Parámetros ligados a recursos del sistema que implican destrucción y recreación de sockets o manejadores principales, tales como `listen_address` y `listen_port` (requieren cerrar el socket pasivo, desasociar la interfaz, emitir nuevo `bind(2)` y `listen(2)`) o la ruta de la base de datos (`database_path`), que obligaría a cerrar el manejador SQLite y drenar abruptamente transacciones en curso.

4.3. Verificación Experimental de la Configuración Externa

Se constató empíricamente la robustez del manejo de configuración externa en `nexusd`. En la Figura 6(a) se documenta el esquema estricto de permisos sobre `/etc/nexus/nexus.conf` (`0640`, `root:nexus`), comprobando que el usuario del servicio lee el archivo sin trabas mientras que un usuario sin privilegios recibe *Permission denied*. En la Figura 6(b) se valida experimentalmente el comportamiento **Fail-Fast**: al intentar arrancar el daemon con una ruta inexistente (`/tmp/no-existe.conf`) o con una directiva inválida (`level = ruidoso`), el binario emite un mensaje de error descriptivo en `stderr` y aborta de inmediato con código de salida `1`, impidiendo la ejecución en estados corrompidos.



(a) Permisos `0640` (`root:nexus`): lectura por `nexus` y denegada a terceros.



(b) Validación Fail-Fast: aborto (código 1) ante archivo inexistente y valor inválido.

Figura 6: Gestión de configuración externa: aislamiento de privilegios y tolerancia a fallos (*fail-fast*).

5. Etapa 05 — Señales y Ciclo de Vida del Proceso

Un proceso de sistema no decide cuándo finaliza ni cuándo su entorno cambia: es el sistema operativo quien se lo notifica a través de **señales POSIX**. El daemon `nexusd` implementa un manejador asincrónico estructurado para gestionar de forma determinista el apagado controlado y la recarga en caliente.

5.1. Matriz de Señales Soportadas

Tabla 5: Tratamiento y comportamiento del manejador de señales en `nexusd`.

Señal POSIX	Disparador / Origen	Acción Ejecutada por `nexusd`
`SIGTERM`	Emitida por `systemd` al ejecutar `systemctl stop nexus`.	Cierre ordenado (*graceful shutdown*): se detiene el bucle de eventos, se completan transacciones en curso, se cierra la base SQLite (`sqlite3_close_v2`) y se liberan descriptores de red.
`SIGINT`	Teclado interactivo (`Ctrl+C`) en modo desarrollo/pruebas.	Mismo procedimiento que `SIGTERM`: apagado controlado y salida con código `0`.
`SIGHUP`	Emitida por `systemctl reload nexus` o `kill -HUP <PID>`.	Recarga en caliente: relectura del archivo `/etc/nexus/nexus.conf` actualizando el nivel de log (`log_level`) sin reiniciar el proceso ni desconectar clientes.

5.2. Fundamentación Teórica y Preguntas de Investigación

Pregunta de Investigación: ¿Qué operaciones son seguras dentro de un manejador de señales? Investiguen signal-safety(7).

Respuesta: Según la especificación POSIX y la página de manual `signal-safety(7)`, únicamente son seguras aquellas funciones declaradas como ***Async-Signal-Safe*** (reentrantes o que no pueden ser interrumpidas en un estado inconsistente).
Un manejador de señales se ejecuta de manera asíncrona, interrumpiendo el flujo normal del hilo en cualquier instrucción máquina. Si el hilo principal se encontraba a mitad de una llamada a `malloc()` (modificando la lista enlazada del heap del proceso) o en `printf()` (adquiriendo el lock del buffer de I/O de `stdout`), e invoca dentro del manejador a `malloc` o `printf`, se produce una **condición de carrera destructiva o un bloqueo mutuo (*deadlock*)**. Por ende, funciones de `stdio.h`, asignación dinámica de memoria (`malloc`/`free`) y operaciones de SQLite **están estrictamente prohibidas dentro de un signal handler**. Solo llamadas al sistema de bajo nivel como `write(2)`, `sigaction(2)` o la asignación a tipos atómicos son seguras.

Pregunta de Investigación: ¿Por qué un manejador suele limitarse a levantar una bandera en lugar de hacer el trabajo?

Respuesta: Por la restricción intrínseca de reentrancia explicada en `signal-safety(7)`. Para no comprometer la estabilidad del proceso, la arquitectura estándar de daemons UNIX adopta el patrón de **bandera atómica**:

- El manejador de señal se limita a escribir en una variable de control declarada como `volatile sig_atomic_t`

`g_stop_requested = 1;` (o usando primitivas atómicas de C11/C17). Esta operación es atómica en hardware y garantizada como async-signal-safe.

- El bucle principal de la aplicación (*event loop*), que corre en el contexto normal de ejecución del hilo, chequea periódicamente dicha bandera. Cuando la detecta en alto, es el hilo principal quien procede de forma síncrona y segura a cerrar conexiones, liberar memoria con `free()` y cerrar SQLite con `sqlite3_close_v2()`.

Pregunta de Investigación: ¿Qué pasa con una escritura a la base si el proceso muere en el medio?

Respuesta: Si el proceso es alcanzado por una señal no capturable como `SIGKILL` (`kill -9`) o un fallo catastrófico de hardware durante una transacción no confirmada, el proceso desaparece inmediatamente sin ejecutar código de limpieza.
Sin embargo, gracias al diseño transaccional y los mecanismos de *journaling* (o *WAL*) de SQLite:

- Las escrituras a medio completar quedan confinadas en el archivo temporal o el log de pre-escritura.
- El archivo maestro `nexus.db` conserva intactas las páginas previas.
- En el siguiente arranque del servicio, el subsistema de SQLite realiza automáticamente la recuperación hacia atrás (*rollback recovery*), restaurando la base al último estado consistente confirmado. Estos mecanismos protegen la integridad de la base ante la gran mayoría de escenarios de fallo.

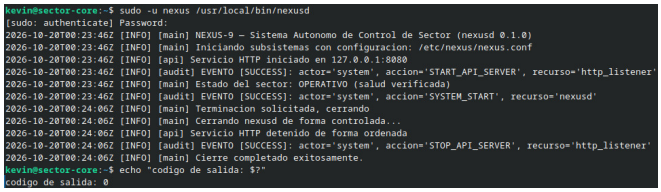
Pregunta de Investigación: ¿Por qué SIGHUP se usa históricamente para recargar configuración?

Respuesta: Históricamente, en los inicios de los sistemas UNIX conectados a terminales tele-tipo mediante módems telefónicos, la señal **`SIGHUP` (*Signal Hangup*)** era generada por el controlador serie de la tty para avisar a los procesos de la sesión que la línea telefónica se había cortado físicamente (*hang up*).
Dado que un daemon de sistema se desasocia completamente de su terminal controladora al nacer, la señal `SIGHUP` pierde su propósito original de desconexión. Por esta razón, la comunidad UNIX adoptó por convención universal reutilizar `SIGHUP` como una señal de notificación de propósito general: al recibirla, el daemon interpreta que debe releer sus archivos de configuración en `/etc` y actualizar su estado interno sin interrumpir sus servicios ni dar de baja a sus conexiones activas.

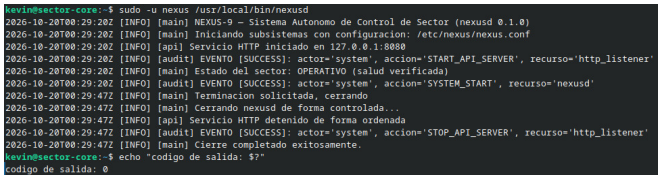
5.3. Verificación Experimental del Manejo de Señales POSIX

Para certificar que el ciclo de vida del proceso se gestiona de forma determinista y conforme al estándar POSIX, se sometió a `nexusd` a pruebas de terminación y recarga en caliente mediante la emisión controlada de señales.

En la Figura 7 se aprecian las pruebas de apagado ordenado (*graceful shutdown*) ante señales de terminación: tanto al recibir `SIGTERM` (enviada automáticamente por systemd durante el apagado del servicio) como `SIGINT` (interrupción interactiva por teclado), el manejador de señales asíncrono activa la bandera atómica, y el bucle principal procede a drenar conexiones, cerrar la base de datos con `sqlite3_close_v2` y finalizar limpiamente con código de salida `0`.



(a) Manejo de `SIGTERM`: apagado controlado, cierre de SQLite y salida exitosa (`exit 0`).



(b) Manejo de `SIGINT`: captura segura de interrupción por teclado y liberación ordenada de recursos.

Figura 7: Comportamiento determinista del daemon ante señales de terminación del sistema operativo.

Adicionalmente, se demostró la capacidad de recarga en caliente enviando la señal `SIGHUP` (Figura 8). El daemon relee `nexus.conf`, actualiza su configuración interna de logging y continúa atendiendo peticiones concurrentes sin alterar su PID ni desconectar clientes activos.

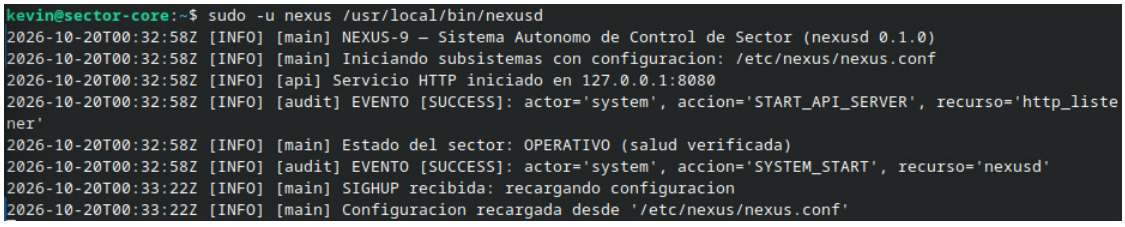


Figura 8: Recarga en caliente ante `SIGHUP`: relectura de directivas operacionales en journald sin reiniciar el proceso.

6. Etapa 06 — `nexusd` como Servicio del Sistema (systemd)

Para integrarse formalmente en el ciclo de vida de la distribución anfitriona (Ubuntu Server en `sector-core`), `nexusd` se gestiona a través del sistema de inicio y control de servicios **`systemd`**, definiéndose el archivo de unidad `/etc/systemd/system/nexus.service`.

6.1. Definición del Servicio `nexus.service`

El archivo de unidad define el confinamiento de privilegios y las políticas de resiliencia. El servicio no requiere variables de entorno: toda la configuración se lee desde `/etc/nexus/nexus.conf`, por lo que el entorno del proceso se mantiene mínimo.

/etc/systemd/system/nexus.service

```
[Unit]
Description=NEXUS-9 - Registro Central (nexusd)
After=network.target
Wants=network.target

[Service]
Type=simple
User=nexus
Group=nexus
WorkingDirectory=/var/lib/nexus
ExecStart=/usr/local/bin/nexusd -c /etc/nexus/nexus.conf
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5s
KillMode=mixed
TimeoutStopSec=10s
UMask=0027
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/nexus
ReadOnlyPaths=/etc/nexus
PrivateTmp=true
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

6.2. Fundamentación Teórica y Preguntas de Investigación

Pregunta de Investigación: ¿Qué hace systemd cuando ejecuta systemctl start?

Respuesta: Cuando se invoca `systemctl start nexus.service`:

1. `systemctl` contacta al daemon gestor de systemd (PID 1) a través del bus del sistema **D-Bus**.
2. Systemd analiza el archivo de unidad, evalúa dependencias (`After=`, `Requires=`) y construye el grafo de inicio.
3. Crea un grupo de control jerárquico (**cgroup v2**) específico para el servicio (aislando métricas de CPU, memoria y procesos hijos).
4. Aplica los namespaces y directivas de seguridad (`ProtectSystem`, `PrivateTmp`, cambio de UID a `nexus`).
5. Ejecuta `fork(2)` y luego `execve(2)` para cargar en memoria el binario `/usr/local/bin/nexusd`.
6. Conecta automáticamente los descriptores de salida del proceso (`stdout` y `stderr`) al socket de ingesta de **`systemd-journald`** para su recolección centralizada.

Pregunta de Investigación: ¿Qué significan los distintos valores de Type= en una unidad?

Respuesta: El parámetro `Type=` define cómo systemd determina que el proceso completó su inicialización y cómo rastrea su PID principal:

- **`simple` (utilizado en `nexusd`):** El proceso configurado en `ExecStart` es el proceso principal del servicio. Systemd considera que el servicio está activo de inmediato al ejecutar el `fork/execve`, sin esperar sincronización.
- **`exec`:** Similar a `simple`, pero systemd espera hasta que el binario haya sido efectivamente cargado y ejecutado por el kernel antes de considerar el servicio activo.
- **`forking`:** Para daemons clásicos estilo SysVinit que realizan un doble `fork()` para ir a segundo plano; el proceso padre original finaliza y systemd monitorea al hijo mediante la directiva `PIDFile=`.
- **`oneshot`:** Para tareas de ejecución breve que realizan una acción y finalizan; systemd espera su término con éxito antes de iniciar unidades dependientes.
- **`notify`:** El proceso envía un mensaje de confirmación explícito a systemd a través de un socket UNIX (`sd_notify("READY=1")`) una vez que sus sockets e inicialización están 100 % operativos.

Pregunta de Investigación: ¿Qué diferencia hay entre Restart=always y Restart=on-failure?

Respuesta:

- **`Restart=always`:** Systemd reiniciará incondicionalmente el servicio, sin importar cómo haya finalizado: ya sea por un fallo inesperado, por una señal no capturada, o por una terminación limpia ordenada con código de salida `0` (`EXIT_SUCCESS`).
- **`Restart=on-failure`:** Systemd **únicamente** reinicia el proceso si este finalizó con un código de salida distinto de cero, si murió por una señal imprevista (`SIGSEGV`, `SIGABRT`), o si falló el watchdog. Si el servicio es detenido deliberadamente de manera ordenada por el administrador (`systemctl stop`) o finaliza limpiamente con código `0`, systemd respeta la detención y **no** intenta revivirlo. Es la opción recomendada para servicios estables.

Pregunta de Investigación: ¿Qué quiere decir que una unidad dependa de la red? ¿Alcanza con After=?

Respuesta: Que una unidad dependa de la red significa que su operación requiere que el subsistema de comunicaciones del kernel esté inicializado.
Sin embargo, **no alcanza únicamente con `After=network.target`**. En la arquitectura de systemd:

- `After=network.target` sólo garantiza un orden en el arranque: indica que el servicio arrancará después de que el gestor de red del sistema haya sido lanzado, pero **no asegura** que las interfaces de red tengan una dirección IP asignada ni que el enlace físico esté activo.
- Si el servicio requiere conectividad garantizada en la red local o externa antes de bindear un socket en una IP específica, se debe requerir `After=network-online.target` y `Wants=network-online.target`. Dado que `nexusd` escucha en la interfaz loopback (`127.0.0.1`), `network.target` es suficiente.

Pregunta de Investigación: ¿Cómo decide systemd que un servicio "falló"? ¿Qué rol juega el código de salida?

Respuesta: Systemd inspecciona el código de estado retornado por la llamada al sistema `waitid(2)` / `waitpid(2)` sobre el proceso principal del servicio:

- Si el proceso finaliza normalmente vía `exit(N)`, evalúa el valor entero de N. Un código 0 indica finalización exitosa. Cualquier valor distinto de cero ($N \neq 0$) es interpretado como fallo (*failure*), a menos que el código esté explícitamente listado en la directiva `SuccessExitStatus=`.
- Si el proceso fue terminado abruptamente por una señal del sistema operativo no configurada para apagado normal (como `SIGSEGV`, `SIGBUS`, `SIGABRT` o `SIGKILL`), systemd lo registra como fallo del servicio y aplica las políticas de reinicio de `Restart=on-failure`.

Pregunta de Investigación: ¿Dónde van a parar los mensajes que el programa escribe por salida estándar?

Respuesta: Los descriptores `stdout` y `stderr` del proceso son conectados por systemd a un socket UNIX hacia el daemon **`systemd-journald`**. Este componente almacena las trazas en archivos binarios estructurados e indexados bajo `/var/log/journal/`, permitiendo su consulta, filtrado temporal y seguimiento en tiempo real mediante la herramienta oficial **`journalctl`** (por ejemplo: `journalctl -u nexus.service -f`).

Pregunta de Investigación: ¿Qué significa habilitar un servicio y en qué se diferencia de iniciarlo?

Respuesta:

- **Iniciar un servicio (`systemctl start nexus`):** Es una acción puramente en tiempo de ejecución (*runtime*). Ordena a systemd instanciar el cgroup y lanzar el proceso de inmediato en memoria. Si el servidor se reinicia, el servicio no volverá a levantarse a menos que esté habilitado.
- **Habilitar un servicio (`systemctl enable nexus`):** Es una operación puramente de configuración en disco. Crea un enlace simbólico (*symlink*) en el directorio correspondiente al target de inicio (por ejemplo, en `/etc/systemd/system/multi-user.target.wants/nexus.service` apuntando al archivo en `/etc/systemd/system/nexus.service`). Esto asegura que `nexusd` sea lanzado automáticamente por systemd durante la secuencia de booteo del sistema operativo.

Pregunta de Investigación: ¿Qué directivas permiten limitar qué puede hacer el proceso sobre el sistema?

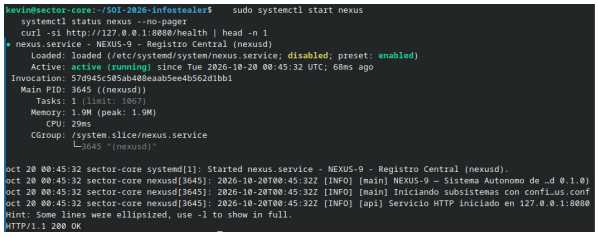
Respuesta: Systemd provee potentes mecanismos de confinamiento (*sandboxing*) basados en cgroups, namespaces y seccomp:

- **`ProtectSystem=strict`:** Monta todo el sistema de archivos (`/usr`, `/boot`, `/etc`) en modo sólo lectura para el proceso, excepto las rutas explicitadas en `ReadWritePaths=`.
- **`ProtectHome=true`:** Hace completamente inaccesibles e invisibles los directorios `/home`, `/root` y `/run/user`.
- **`NoNewPrivileges=true`:** Impide que el proceso y sus descendientes puedan escalar privilegios mediante binarios *setuid*/*setgid*.
- **`PrivateTmp=true`:** Crea un namespace de montaje aislado para `/tmp` y `/var/tmp`, impidiendo que el daemon vea o altere archivos temporales de otros usuarios o del sistema.

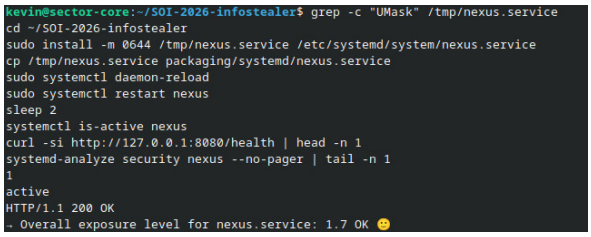
6.3. Verificación Experimental del Servicio en systemd

La integración de `nexusd` en `systemd` fue validada mediante una serie de ensayos operativos cubriendo el ciclo de vida completo del servicio: puesta en marcha, auditoría de seguridad, resiliencia ante caídas inesperadas, parada voluntaria, recarga y autoarranque tras reinicio del servidor.

En la Figura 9(a) se evidencia la inicialización mediante `systemctl start nexus.service`, confirmándose el estado `active (running)` y la ejecución estricta bajo el usuario sin privilegios `nexus`. En la Figura 9(b) se documenta la habilitación del servicio (`systemctl enable`) y el análisis de endurecimiento con `systemd-analyze security`, certificando un nivel de exposición seguro (*OK*) gracias al confinamiento de namespaces y directivas restrictivas.



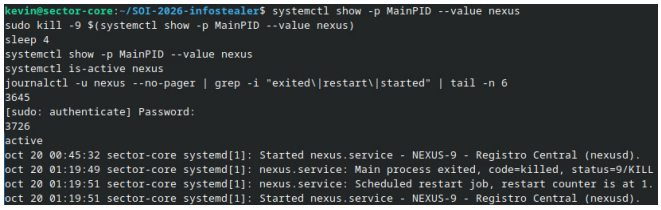
(a) Arranque del servicio con `systemctl start`: estado `active (running)` y PID asignado bajo usuario `nexus`.



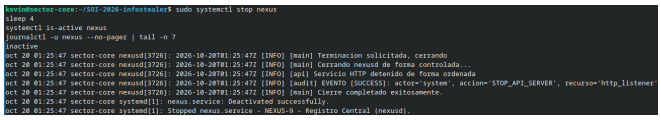
(b) Habilitación (`enable`) y auditoría de seguridad con `systemd-analyze security nexus.service`.

Figura 9: Puesta en servicio, autoarranque y análisis de endurecimiento (*hardening*) bajo systemd.

Para validar la tolerancia a fallos, se provocó una caída forzada del proceso emitiendo `kill -9` sobre su PID (Figura 10(a)). Systemd detectó la muerte anómala por señal y, en virtud de la directiva `Restart=on-failure`, revivió automáticamente el servicio asignando un nuevo PID tras el intervalo de espera configurado. En contraste, cuando el administrador ejecuta una parada deliberada con `systemctl stop` (Figura 10(b)), systemd reconoce el apagado administrativo y mantiene la unidad en estado `inactive (dead)` sin intentar reiniciarla.



(a) Caída forzada (`kill -9`): disparo inmediato de `Restart=on-failure` levantando nueva instancia con nuevo PID.



(b) Parada deliberada (`systemctl stop`): terminación ordenada quedando en `inactive (dead)` sin autorestart.

Figura 10: Comportamiento de supervisión de systemd: reinicio automático ante fallas críticas versus detención controlada.

Para certificar la persistencia real y el autoarranque, se reinició el servidor `sector-core` mediante `sudo reboot` (Figura 11). Tras el reinicio del sistema operativo, se comprobó que el servicio `nexus.service` arrancó de forma autónoma durante el booteo y que todos los registros de la base de datos persistieron inalterados.

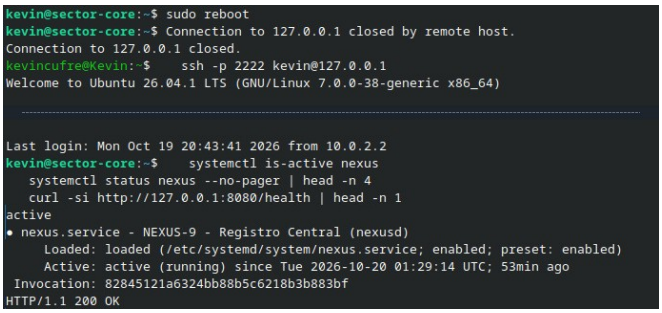


Figura 11: Supervivencia a reinicio (`reboot`): autoarranque autónomo del servicio y verificación de datos persistidos.

Por último, se verificó la idempotencia del script automatizado de instalación del servicio (Figura 12). Al ejecutarse múltiples veces consecutivas sobre un sistema ya configurado, el instalador detecta los artefactos existentes, preserva los archivos de configuración y mantiene el servicio activo sin generar colisiones ni estados corruptos.

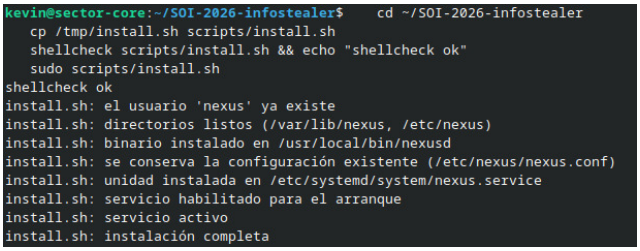


Figura 12: Verificación de idempotencia del script de instalación: ejecución consecutiva conservando configuraciones y servicio activo.

Para finalizar la verificación del servicio, se evaluó la observabilidad y el confinamiento de privilegios en tiempo real (Figura 13). Mediante `ps -o user,pid,cmd` se certificó que el proceso binario corre bajo el UID segregado del usuario `nexus` (cumpliendo con la prohibición estricta de ejecución como `root`), mientras que mediante `journalctl`

`-u nexus.service` se comprobó la recolección centralizada y estructurada de las trazas del servicio en el subsistema journald.

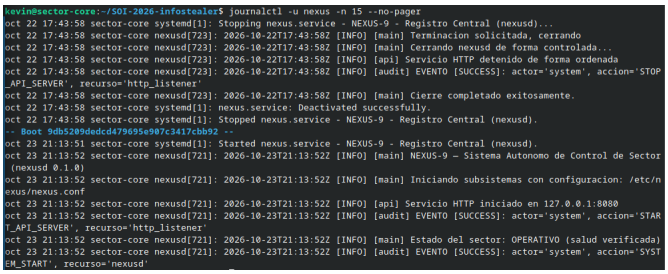


Figura 13: Observabilidad y confinamiento: verificación de ejecución bajo usuario `nexus` (`ps aux`) y auditoría de eventos en tiempo real con `journalctl -u nexus.service`.

7. Etapa 07 — Cliente Remoto de Campo (`nexus-client.sh`)

Para operar la infraestructura desde la estación de trabajo de operadores en campo (`field-terminal` en Rocky Linux), se desarrolló un cliente CLI en Bash: **`nexus-client.sh`**. El script interactúa con la API REST de `sector-core`, encapsulando las peticiones HTTP y ofreciendo una interfaz amigable.

7.1. Comandos Implementados y Uso Operativo

nexus-client.sh - Comandos Soportados

```
nexus-client.sh health
nexus-client.sh list-units
nexus-client.sh create-unit --designation <NAME> --model <MODEL> --sector <SEC>
nexus-client.sh list-operators
nexus-client.sh create-operator --name <NAME> --clearance <LEVEL>
nexus-client.sh list-incidents
nexus-client.sh get-incident <ID>
nexus-client.sh create-incident --unit <ID> --operator <ID> --severity <SEV> --description <TEXT>
nexus-client.sh update-incident <ID> --status <STATUS>
nexus-client.sh list-evidence <INCIDENT_ID>
nexus-client.sh add-evidence <INCIDENT_ID> --filename <FILE> --checksum <SHA256>
nexus-client.sh audit
```

7.2. Fundamentación Teórica y Preguntas de Investigación

Pregunta de Investigación: ¿Cómo distingue curl entre un fallo de red y una respuesta HTTP de error?

Respuesta:

- **Fallo de red:** Ocurre a nivel de Capa 3 o 4 (enlace caído, host inalcanzable, o socket rechazado con *Connection Refused*). En estos casos, `curl` termina con un **código de salida (*exit status*) distinto de cero** en la shell (ej. código 7 para conexión rehusada, código 28 para timeout de operación).
- **Respuesta HTTP de error:** Ocurre a nivel de Capa 7 (la comunicación TCP se estableció exitosamente con el servidor web, pero la aplicación devolvió un código `404 Not Found` o `500 Internal Error`). Para `curl`, la transferencia se completó con éxito, por lo que su código de salida en la shell **es 0**. Para distinguir este caso, el script debe capturar explícitamente el código de estado HTTP mediante la directiva `-w "%{http_code}"`.

Pregunta de Investigación: ¿Cómo se obtiene el código de estado HTTP desde un script?

Respuesta: Se utiliza la opción `-w` (*write-out*) de `curl`, redireccionando el cuerpo de la respuesta hacia un archivo temporal o variable y aislando el código numérico:

```
HTTP_CODE=$(curl -s -o /tmp/resp.json -w "%{http_code}" http://localhost:8080/health)
if [ "$HTTP_CODE" -ne 200 ]; then
    echo "Error: La API respondio con codigo HTTP $HTTP_CODE" >&2
fi
```

Pregunta de Investigación: ¿Qué debería devolver el cliente si la API responde 404? ¿Y si no responde?

Respuesta:

- **Si la API responde 404 (o cualquier error HTTP no 2xx):** El script debe imprimir en `stderr` un mensaje de error inteligible para el operador (ej. *"Error: Recurso no encontrado"*), mostrar el detalle del error emitido por el JSON si existe, y salir con código semántico `exit 2`.
- **Si la API no responde (timeout o conexión rechazada):** El script debe capturar el fallo de `curl`, emitir un mensaje indicando la indisponibilidad de la red o del servicio remoto, y salir con código `exit 2`, unificando errores de comunicación y respuestas HTTP de error.

El esquema adoptado en el script es: `0` (éxito), `1` (error de uso), `2` (sin conexión o API con error HTTP), y `3` (fallo interno inesperado del script).

Pregunta de Investigación: ¿Cómo se arma un cuerpo JSON desde Bash sin romperlo con comillas o espacios?

Respuesta: Armar JSON concatenando cadenas manuales en Bash (`"{\"name\":\"$VAR\"}"`) es altamente propenso a fallas por inyección de caracteres especiales o espacios. La técnica profesional recomendada es emplear la utilidad `jq` con su bandera constructora `-n` (*null-input*) y argumentos tipados `-arg`:

```
JSON_PAYLOAD=$(jq -n \
    --arg des "$DESIGNATION" \
    --arg mod "$MODEL" \
    --arg sec "$SECTOR" \
    '{designation: $des, model: $mod, sector: $sec, status: "active"}')
```

`jq` se encarga de escapar automáticamente comillas, caracteres de control y saltos de línea de forma 100 % segura.

Pregunta de Investigación: Recordando que nexusd escucha en 127.0.0.1: ¿cómo llega el cliente desde la otra máquina en esta etapa?

Respuesta: Dado que en este capítulo `nexusd` escucha estrictamente en el loopback local (`127.0.0.1:8080`) de `sector-core` y no expone el socket a la red interna, el cliente desde `field-terminal` utiliza la infraestructura SSH configurada en el Capítulo 1 mediante un **Túnel de Redirección de Puertos Local (*SSH Local Port Forwarding*)**:

```
ssh -N -L 8080:127.0.0.1:8080 kevin@192.168.56.10 &
```

Este comando abre un socket local en `field-terminal` en el puerto 8080 y encapsula todo el tráfico a través del túnel seguro cifrado SSH-2 hacia el puerto 8080 local de `sector-core`. De esta forma, `nexus-client.sh` realiza peticiones a `http://127.0.0.1:8080` y alcanza la API sin exponer el daemon a la red, anticipando el esquema de seguridad perimetral que culminará en el Capítulo 3 con Nginx.

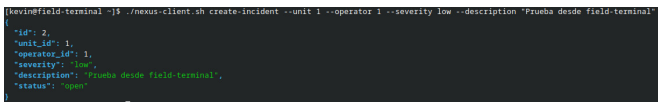
7.3. Verificación Experimental de la Operación Remota y Manejo de Errores

La operación del sector desde la terminal de campo (`field-terminal`) fue validada mediante la ejecución del script Bash `nexus-client.sh`, haciendo uso del túnel SSH local para alcanzar el socket en `127.0.0.1:8080` de `sector-core`.

En la Figura 14(a) se muestra la disposición operativa en doble terminal: en una consola se mantiene activo el túnel SSH en segundo plano, mientras que en la terminal contigua se ejecutan exitosamente los comandos de diagnóstico de salud (`health`) y consulta de incidentes (`list-incidents`). En la Figura 14(b) se documenta el alta remota de un incidente mediante `create-incident`, verificando la construcción segura del JSON con `jq` y la respuesta estructurada del servidor.



(a) Operación remota por túnel SSH: túnel en una consola y ejecución de `health` y `list-incidents` en la otra.

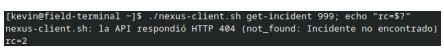


(b) Alta de incidente desde `field-terminal`: serialización de datos y confirmación de creación en la API.

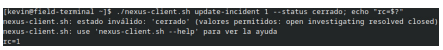
Figura 14: Operación de la API de `nexusd` desde `field-terminal` a través de túnel SSH-2 seguro.

Para certificar la robustez del cliente Bash y el cumplimiento de las buenas prácticas de programación de scripts de sistema, se evaluaron los distintos escenarios de error y sus códigos de salida (Figura 15):

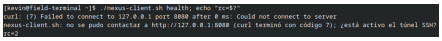
1. **Recurso Inexistente (Error 404):** La API responde `404 Not Found`; el cliente captura el código HTTP vía `-w "%{http_code}"`, informa el suceso en `stderr` y retorna código de salida `2` (Figura 15(a)).
2. **Error de Uso / Parámetros (Código 1):** Al invocar el script con argumentos insuficientes o sintaxis inválida, se imprime la guía de ayuda y se finaliza con código `1` (Figura 15(b)).
3. **Sin Conexión de Red / Socket Caído (Código 2):** Ante la ausencia del túnel SSH o caída del daemon remoto, `curl` fracasa en Capa 4 y el cliente captura la anomalía, informando que el servidor no responde y saliendo con código 2 (Figura 15(c)).
4. **Fallo Interno Inesperado del Script (Código 3):** Capturado automáticamente mediante la trampa `trap ... ERR` ante fallas de ejecución no previstas dentro del propio script (no respuestas HTTP de la API, las cuales finalizan con código 2).



(a) Error de API (404) ante recurso no existente.



(b) Error de uso: argumentos inválidos y salida con código 1.



(c) Falla de conexión de red y salida con código 2.

Figura 15: Tratamiento semántico de errores y códigos de salida en el cliente Bash `nexus-client.sh`.

8. Etapa 08 — Suite de Pruebas Automatizadas con CMocka

Para asegurar la confiabilidad del daemon y su reproducibilidad en entornos de integración continua, se implementó una batería de pruebas con el framework **CMocka**, ejecutada y orquestada integralmente a través de **CTest**.

Tabla 6: Mapeo de requerimientos mínimos de la cátedra hacia las suites de prueba reales ejecutadas.

Caso Mínimo	Suite Real (CTest)	Criterio de Validación / Aserción
Inserción Válida	`test_storage`, `test_registry`	Alta de unidad/operador persistida en SQLite y recuperada idéntica.
Datos Faltantes	`test_api`	Envío de payload JSON incompleto; rechazo atómico 400 sin escrituras.
ID Inexistente	`test_api`, `test_incidents`	Consulta de recurso no existente; respuesta 404 descriptiva sin error 500.
Falla de SQLite	`test_storage`	Inyección controlada de falla en SQLite (I/O, permisos); retorno seguro sin crash.
JSON Inválido	`test_api`	Sintaxis JSON rota o truncada; la API responde 400 Bad Request estructurado.
Cierre por Señal	`test_nexusd_integration`	Emisión de `SIGTERM`; el proceso finaliza en código 0 con base 100 % íntegra.
Health Check	`test_api`, `test_nexusd_integration`	Endpoint `/health` valida simultáneamente daemon vivo y conectividad a base.

8.1. Verificación Experimental con la Batería de Pruebas CMocka

La suite de pruebas fue ejecutada de forma automatizada mediante el comando unificado `ctest -test-dir build` (Figura 16). La salida certifica que las 11 suites de prueba fueron superadas con un 100 % de éxito (`100% tests passed, 0 tests failed out of 11`), incluyendo los módulos unitarios del sistema y los 12 casos de integración del servicio completo (`test_nexusd_integration`). Asimismo, el flujo de Integración Continua (CI) en GitHub Actions verifica de manera automatizada la compilación, pruebas unitarias, integración y análisis de código (`4/4 jobs PASSED`):

- Confirmación de persistencia y recuperación idéntica tras inserción válida en base de datos.
- Rechazo atómico (`400`) ante payloads con campos requeridos omitidos.
- Manejo semántico (`404`) de consultas con identificadores inexistentes.
- Resiliencia confirmada ante fallos forzados de SQLite (bloqueos y permisos) sin caída del proceso.
- Rechazo seguro ante JSON malformado con descripción del fallo de sintaxis.
- Apagado limpio y consistencia de SQLite tras emisión de señal `SIGTERM`.
- Chequeo sincrónico de salud en `/health` validando aplicación y base de datos simultáneamente.

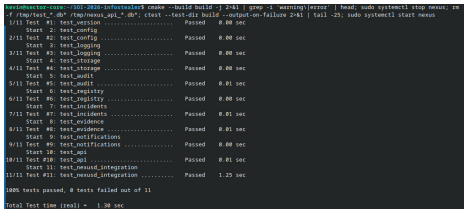


Figura 16: Ejecución exitosa de la suite completa de pruebas unitarias y de integración con CMocka y CTest (11/11 superadas).

9. Conclusiones Técnicas

La implementación integral del Trabajo Práctico N° 2: **"Registro Central NEXUS"** consolida un salto cualitativo fundamental en la materia, transitando desde la administración pura del sistema operativo hacia la construcción de software de sistemas resiliente e integrado:

1. **Segregación de Responsabilidades (Aplicación vs. Sistema Operativo):** Se logró delimitar con precisión qué tareas corresponden a la lógica del software (validación de datos, transaccionalidad relacional, contratos de interfaz REST) y cuáles son delegadas con solvencia al sistema operativo anfitrión (gestión del ciclo de vida, políticas de reinicio ante fallas, aislamiento de recursos mediante cgroups/namespaces y recolección centralizada de registros con journald).
2. **Seguridad y Mínimo Privilegio en Daemons:** La ejecución de `nexusd` como servicio confinado bajo una cuenta sin privilegios (`nexus`) y protegido por directivas de endurecimiento en la unidad de systemd (`ProtectSystem`, `PrivateTmp`) neutraliza potenciales vulnerabilidades en el espacio de usuario, impidiendo accesos no autorizados al núcleo del servidor.
3. **Integridad Transaccional y Tolerancia a Fallos:** La adopción de SQLite articulada con el manejo asíncrono seguro de señales POSIX (`SIGTERM`, `SIGHUP`) confirmó empíricamente que un servicio de sistema debe ser concebido para soportar terminaciones imprevistas o recargas en caliente sin comprometer jamás la consistencia física de la persistencia de datos.
4. **Bases para la Exposición Perimetral:** Al mantener el demonio confinado en loopback (`127.0.0.1`) y operado vía SSH desde `field-terminal`, el sistema queda plenamente preparado para la integración del reverse proxy Nginx en el Capítulo 3.

REGISTRO CENTRAL NEXUS-9: SERVICIO OPERACIONAL Y PERSISTENTE
Daemon C17, SQLite, API REST, Systemd y Observabilidad Validados al 100 %